CS3000 Practice Quiz 5B — Answer Key

0/1 knapsack and edit distance

Notation follows the class notes on dynamic programming: $C(v)$ (coin change), $K(i, w)$ (0/1 knapsack), $L(i, j)$ (LCS), $D(i, j)$ (edit distance); strings and items are 1-indexed. Every table below was computed by a program and checked against brute force. Shaded cells mark the traceback path. Point splits imitate the Gradescope rubrics used on Quizzes 1–4 (each question 5 pts, 10 total).

Q1. 0/1 knapsack, $W = 7$

(a)

$$K(i,w) = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ K(i-1, w) & w_i > w \\ \max\big(K(i-1,w),\ v_i + K(i-1, w-w_i)\big) & w_i \le w \end{cases}$$

(b) Table (shaded = traceback path).

$i\downarrow\ w\rightarrow$	0	1	2	3	4	5	6	7
$i = 0$	**0**	0	0	0	0	0	0	0
$i = 1$	**0**	1	1	1	1	1	1	1
$i = 2$	0	1	1	**4**	5	5	5	5
$i = 3$	0	1	1	4	5	6	6	**9**
$i = 4$	0	1	1	4	5	7	8	**9**

(c) Traceback. $K(4, 7) = 9 = K(3, 7)$ so package 4 is *not* used. $K(3, 7) = 9 \neq K(2, 7) = 5$, so package 3 is used; $w = 7 - 4 = 3$. $K(2, 3) = 4 \neq K(1, 3) = 1$, so package 2 is used; $w = 3 - 3 = 0$. Optimal load: **packages 2 and 3**, weight 7, value **9** (the unique optimum).

(d) Greedy fails. Ratios: $1/1 = 1$, $4/3 \approx 1.33$, $5/4 = 1.25$, $7/5 = 1.4$. Greedy loads package 4 (weight 5), then package 2 does not fit (8 kg), package 3 does not fit, then package 1: total value $7 + 1 = \mathbf{8} < 9$.

(e) $\Theta(nW)$ time (one $O(1)$ cell per (i, w)) and $\Theta(nW)$ space for the full table (needed for the traceback).

Rubric (5 pts).

- **+1 pt** Recurrence with both base cases and the $w_i > w$ case.
- **+1.5 pts** Table correct (-0.5 per wrong row).
- **+1 pt** Traceback identifies packages 2 and 3 (value 9).
- **+1 pt** Greedy counterexample computed (value 8).
- **+0.5 pts** $\Theta(nW)$ time and space.

Common mistakes: letting an item be reused (that is the coin-change/unbounded recurrence); comparing with $K(i, w - 1)$ instead of $K(i - 1, w)$ in the traceback.

Q2. Edit distance `CART` → `MARCH`

(a)

$$D(i,j) = \begin{cases} j & i = 0 \\ i & j = 0 \\ D(i-1, j-1) & A[i] = B[j] \\ 1 + \min\big(D(i, j-1),\, D(i-1, j),\, D(i-1, j-1)\big) & A[i] \neq B[j] \end{cases}$$

($D(i, j-1)$: insert $B[j]$; $D(i-1, j)$: delete $A[i]$; $D(i-1, j-1)$: substitute.)

(b) Table (shaded = traceback path).

$A_i \downarrow$ $B_j \rightarrow$	0	1 (M)	2 (A)	3 (R)	4 (C)	5 (H)
0	**0**	1	2	3	4	5
1 (C)	1	**1**	2	3	3	4
2 (A)	2	2	**1**	2	3	4
3 (R)	3	3	2	**1**	**2**	3
4 (T)	4	4	3	2	2	**3**

(c) $D(4, 5) = \mathbf{3}$. One optimal sequence from the traceback: substitute `C`→`M`; keep `A`; keep `R`; insert `C`; substitute `T`→`H`. That is: `CART`→`MART`→`MARCT`→`MARCH`. Other 3-operation sequences exist; any valid one earns full credit.

Rubric (5 pts).

+1 pt Recurrence including both base cases and the free match.

+2.5 pts Table correct (−0.5 per wrong row).

+1.5 pts Distance 3 and a valid 3-operation sequence from the table.

Common mistakes: forgetting the base row/column ($D(0, j) = j$, $D(i, 0) = i$); adding 1 on a match; mixing up which neighbor means insert vs. delete.